GGSEC RESEARCH · WHITEPAPER

Firmware Persistence Techniques

Advanced UEFI & embedded persistence analysis — attack methodologies, forensic visibility, and platform integrity risk

Author: Maciej Gojny

Organization: GG Advanced IT Security

Web: ggsec.de

Version: 1.2 · July 2026

TLP:WHITE — PUBLIC RELEASE

00 Executive Abstract

This whitepaper provides a technical overview of firmware-based persistence techniques observed in modern cyber operations targeting UEFI firmware, embedded systems, storage controllers, and boot environments. Unlike traditional operating system persistence, firmware-level mechanisms may survive operating system reinstallation, disk replacement, and conventional remediation procedures. The document focuses on real-world attack methodologies, defensive strategies, forensic visibility, and the platform integrity risks associated with modern firmware ecosystems.

> **Responsible disclosure:** certain low-level offensive implementation details have intentionally been omitted. This document is intended for defensive security, threat modeling, and firmware integrity assessment purposes only.

Contents

01 Introduction & Scope

Firmware persistence represents one of the most sophisticated categories of long-term system compromise in modern cybersecurity operations. While conventional malware operates within user-space or kernel-space boundaries, firmware-based threats target low-level platform initialization, boot-chain integrity, and embedded hardware logic. These persistence mechanisms may remain active even after disk formatting, operating system reinstallation, or complete storage replacement.

This document consolidates publicly documented attack methodologies, persistence mechanisms, notable threat campaigns, forensic indicators, and defensive mitigations relevant to enterprise and embedded security environments. It is intended for defensive security practitioners, firmware auditors, and enterprise security architects.

02 UEFI Fundamentals

2.1 What is UEFI?

UEFI (Unified Extensible Firmware Interface) defines the interface between platform firmware and the operating system boot process, succeeding traditional BIOS architectures. Modern UEFI implementations use modular firmware components stored in SPI flash memory soldered directly to the motherboard. Because firmware operates independ-

ently from the operating system and storage media, compromise at this layer fundamentally undermines platform trust assumptions.

2.2 UEFI Boot Process — Complete Phase Reference

The full UEFI boot sequence consists of six phases. All six must be considered in firmware threat modeling.

Phase	Full Name	Firmware role	Attack surface
SEC	Security	Root-of-trust, Cache-as-RAM init	Integrity violation; unauthorized code execution before the root of trust is established
PEI	Pre-EFI Init	Hardware init, memory detection, DRAM training	Malicious PEIMs
DXE	Driver Exec Env	Loads firmware drivers, publishes protocols	Malicious DXE driver insertion; modification of existing components (MoonBounce)
BDS	Boot Device Sel	Boot policy, device enumeration, boot manager	Boot order manipulation, WPBT abuse
TSL	Transient Sys Load	OS loader execution, ExitBootServices() handoff	Bootkit injection prior to ExitBootServices (BlackLotus)
RT	Runtime Services	UEFI runtime services available post-boot (SetVariable)	Persistent NVRAM variable abuse

> **Note:** TSL and RT are frequently omitted from vendor reference material but represent critical attack surface. TSL is the execution window exploited by bootkits prior to kernel handoff.

03 Firmware Persistence Techniques

3.1 UEFI/BIOS Firmware Modification

Attackers modify firmware startup logic to execute malicious components before operating system initialization. A malicious UEFI DXE driver is inserted into the firmware image. Some variants embed an NTFS driver enabling write access to the OS partition directly from firmware, allowing malware to be re-dropped after OS reinstallation. This persistence survives disk formatting and storage replacement.

3.2 Windows Platform Binary Table (WPBT) Abuse

WPBT is an ACPI table used by firmware to pass a PE executable to Windows for execution during early boot. Microsoft requires digital signing but does not validate certificate revocation status by default. It has been available as an attack surface since Windows 8 (October 2012).

Property	Detail
Secure Boot bypass required	No — WPBT runs within OS context after boot
Code execution level	Kernel-level at startup via Session Manager
Persistence method	Binary dropped to disk, re-executed at each boot
Signing enforcement	Digital signature required; revocation not checked by default
Mitigation	WDAC policy restricting unsigned or revoked PE binaries

3.3 SSD/HDD Firmware Modification

Modified storage controller firmware can maintain persistence independently of the file system. Demonstrated capabilities include hidden Host Protected Area (HPA) or Device Configuration Overlay (DCO) storage invisible to the OS, boot-time code injection from the controller, and survival of full disk format and partition table erasure. This typically requires physical access or supply-chain compromise.

3.4 UEFI Variable / NVRAM Persistence

UEFI NVRAM variables accessible via GetVariable/SetVariable runtime services can store attacker-controlled data or shellcode read by a malicious DXE driver at next boot. This technique requires existing ring-0 access to write variables but provides stealthy persistence that survives OS reinstallation.

3.5 OEM Persistence Modules — The Legitimate Backdoor Class

A category distinct from malware deserves separate treatment: vendor-supplied firmware modules that implement the same technical primitives as an implant but ship from the factory. They are legitimate by intent and largely indistinguishable from an implant by behaviour. Threat actors have repurposed them directly.

Module	Mechanism	Security relevance
Computrace / LoJack Absolute Software	UEFI module drops `rpcnetp.exe` to System32 by replacing `AUTOCHK.EXE`; installs a Windows service; C2 to vendor infrastructure	**Repurposed by APT28 as LoJax — the first in-the-wild UEFI rootkit. Factory-embedded on a reported 600M+ devices.**
Lenovo Service Engine Lenovo	WPBT-based; replaces `autochk.exe` to reinstate `LenovoUpdate.exe` / `LenovoCheck.exe` after a clean OS install	Withdrawn 2015 after buffer overflow disclosure; removal required a BIOS update
App Center D&I GIGABYTE	`WpbtDxe.efi` drops an embedded Windows native binary that fetches payloads over plain HTTP with no signature validation	**271 board models, ~7M devices (Eclypsium, 2023). MITM-exploitable by design.**
Password recovery paths ASUS, HP, others	Undocumented service mechanisms permitting UEFI setup or HDD password reset without authentication	Patched individually; illustrates the class rather than any single case

> **Detection guidance:** the reliable structural signal is a Windows-subsystem PE (Subsystem 1 = Native, or 2 = GUI) embedded inside a UEFI-subsystem module (Subsystem 10–12). Supporting indicators: references to the WPBT ACPI table; the strings `autochk.exe`, `AUTOCHK.BAK`, `BootExecute`; and plain-HTTP update URLs inside DXE modules.

Assessment note: a dormant OEM module is not an active compromise, but it is not zero-risk either. Where SPI descriptor access control permits host writes to the BIOS region, an attacker with local privilege can reactivate or replace such a module. Report the descriptor permission finding as the primary issue and the OEM module as context.

04 UEFI Bootkits & Advanced Threats

UEFI bootkits target boot-chain integrity and manipulate EFI components, boot managers, or OS loading behaviour before kernel initialization. They represent the most technically sophisticated category of firmware persistence.

4.1 Bootkit versus Firmware Implant

These two terms are frequently conflated in public reporting. The distinction determines remediation cost and must be established before an incident response plan is written.

Property	Bootkit (ESP-resident)	Firmware implant (SPI-resident)
Storage location	EFI System Partition on disk	SPI flash on the motherboard
Survives OS reinstall	Yes, if the ESP is not wiped	Yes
Survives disk replacement	**No**	**Yes**
Write protections in path	None — ESP is a normal FAT32 partition	BIOS_CNTL, PRx, SMM protections must be defeated
Remediation	Secure ESP wipe and OS reinstall	Verified reflash or board replacement
Examples	BlackLotus, ESPecter, Bootkitty	LoJax, MosaicRegressor, MoonBounce, CosmicStrand

4.2 BlackLotus

BlackLotus is the first publicly documented UEFI bootkit capable of bypassing Secure Boot on fully patched Windows 11 systems under specific conditions. It exploits **CVE-2022-21894 (“Baton Drop”)**, a Secure Boot Security Feature Bypass in signed copies of the Windows Boot Manager: the vulnerable boot manager truncates the Secure Boot policy values and continues booting instead of halting on missing DB/DBX data.

- Leverages a legitimate, validly Microsoft-signed boot manager that remains trusted by the Secure Boot DB
- **Downgrade attack:** the old bootloader stays trusted despite the known vulnerability, because it was never added to the revocation list
- Establishes persistence by enrolling the attacker’s Machine Owner Key (MOK)
- Disables HVCI, BitLocker and Microsoft Defender Antivirus
- Deploys an unsigned kernel driver plus a user-mode HTTP downloader for C2
- Advertised on underground forums from October 2022 at approximately US$5,000; first public analysis by ESET, March 2023
- **Not an initial access vector** — deployment requires prior privileged or physical access

Related but distinct: CVE-2023-24932 is a further Secure Boot bypass addressed by Microsoft in May 2023. Its fix required manual opt-in activation, which significantly delayed real-world protection. Both CVEs appear in BlackLotus discussions; only CVE-2022-21894 is the vulnerability the bootkit exploits.

Remediation scope: BlackLotus persists on the ESP, not in SPI flash. Physical reflash is *not* required. Treat ESP wipe plus OS reinstall as sufficient, and reserve reflash for confirmed SPI-resident implants.

4.3 Downgrade Attack Technique

Secure Boot validates cryptographic signatures but does not enforce version currency by default. An attacker substitutes a vulnerable but validly signed older boot component, rolling back platform security to an exploitable state. Secure Boot DBX revocation updates are the primary mitigation but require active deployment by administrators. Properly maintained Secure Boot with a current DBX remains effective — BlackLotus does not represent a universal Secure Boot bypass.

05 Detection & Forensics

5.1 Indicators of Compromise

IOC type	Description	Priority
ESP modifications	Unexpected .efi binaries in the EFI System Partition; modified BCD store entries or boot order; mismatched file modification timestamps versus `C:\Windows\Boot\EFI`	**P1 — Critical**
Secure Boot changes	Secure Boot disablement; tampered DBX/DB entries; Secure Boot audit log anomalies	**P1 — Critical**
HVCI / BitLocker disabled	Sudden deactivation of Memory Integrity or BitLocker without an administrative change record	**P1 — Critical**
NVRAM variable changes	Unexpected creation or modification of UEFI variables, especially BootOrder and Boot####	P2 — High
Hardware anomalies	Firmware version mismatch versus baseline; inconsistent hardware telemetry; unexpected PCI device	P2 — High
Unknown DXE modules	Unsigned or unrecognized DXE modules present in the firmware image versus a known-good baseline	P2 — High
Modified existing modules	Byte-level divergence in a pre-existing component such as CORE_DXE, with no new files on the ESP	P2 — High

5.2 Detection & Analysis Tools

Tool	Primary function	Operational note
ggfw GGSEC	Automated offline SPI analysis: Intel Flash Descriptor access control, UEFI FV/FFS walking, PE/DXE anomaly heuristics, variable-store analysis, PE/TE extraction for YARA triage, bounded disassembly	Single-pass posture assessment with confidence-scored findings; heuristic triage, not a signature engine; live MEI fallback where offline evidence is insufficient
CHIPSEC	Firmware security auditing, SPI flash analysis, UEFI variable inspection	Requires a kernel driver; may require Secure Boot disable in some configurations — assess risk before production use
UEFITool	UEFI firmware image parsing, module extraction, section inspection	Safe for offline analysis; no driver required
Binwalk	Firmware extraction, entropy analysis, file carving	Effective for embedded/IoT firmware; limited for complex UEFI images
Ghidra	Reverse engineering of DXE drivers, PEI modules, UEFI PE binaries	Use community UEFI loader scripts for correct loader support
fwupd / LVFS	Linux firmware update and integrity baseline tracking	Useful for firmware version drift detection across a fleet

5.3 Sigma Rule — Suspicious EFI File in ESP

Both conditions are required. Omitting `SignatureStatus` generates excessive false positives on every legitimate firmware update.

```
title: Suspicious EFI File in ESP
id: a1b2c3d4-e5f6-7890-abcd-ef1234567890
status: experimental
description: Detects unsigned .efi binaries written to EFI System Partition
references:
  - https://attack.mitre.org/techniques/T1542/003/
author: GGSEC Research
logsource:
  product: windows
  service: sysmon
detection:
  selection:
    EventID: 11
    TargetFilename|contains: '\EFI\'
    SignatureStatus: 'unsigned'
  condition: selection
falsepositives:
  - Legitimate firmware update tools writing signed EFI components
  - OEM provisioning software
level: high
tags:
  - attack.persistence
  - attack.t1542.003
```

5.4 Splunk Query — Secure Boot Tampering

```
index=windows sourcetype=WinEventLog:System EventID=1035
| eval change_detail=coalesce(Param1, "unknown")
| where like(change_detail, "%SecureBoot%") OR like(change_detail, "%BitLocker%")
| stats count by ComputerName, change_detail, _time
| sort - _time
```

Supplement with `WinEventLog:Security` EventID 4616 and `Microsoft-Windows-Kernel-Boot/Operational` for comprehensive boot integrity coverage.

5.5 SPI Acquisition Caveat

> A host-side SPI dump may return a blank ME region where descriptor permissions deny the read (FRAP/BRRA). **Full image size does not imply full image content.** Verify region coverage and blank ratios before asserting anything about ME version or integrity from an offline dump; use a live MKHI query instead. Physical acquisition with a hardware programmer avoids the issue entirely.

06 Defensive Mitigations

Effective defense against firmware persistence requires layered platform security. No single control is sufficient; combine hardware root-of-trust technologies, Secure Boot validation, firmware integrity monitoring, restrictive execution policies, and incident response procedures that specifically address firmware compromise scenarios.

6.1 Platform Security Controls

Control	Description	Threat covered
Secure Boot	Only cryptographically signed EFI components permitted during the boot chain	Bootkits, unauthorized DXE drivers
TPM 2.0	Measures firmware integrity (PCR 0–7) during boot; seals the BitLocker key to expected platform state	Firmware tampering detection
Intel Boot Guard	Hardware-enforced verification of the Initial Boot Block (IBB) before firmware execution	Pre-SEC firmware modification
AMD PSP	Platform Security Processor — hardware root of trust equivalent to Intel Boot Guard	Pre-SEC firmware modification
HVCI / Memory Integrity	Prevents unsigned kernel drivers from loading at runtime via hypervisor enforcement	BlackLotus kernel driver stage
SPI write protection	BIOS_CNTL lock, Protected Range registers, descriptor access control audit	SPI-resident implant installation
EDR with firmware scanning	Endpoint agents with firmware telemetry and change detection	Post-compromise detection
Platform Firmware Resilience	Detect, protect and recover firmware integrity per NIST SP 800-193	Critical infrastructure

6.2 WDAC for WPBT Mitigation

Microsoft recommends Windows Defender Application Control to restrict binaries delivered via WPBT. A restrictive WDAC policy prevents unsigned or policy-non-compliant binaries from executing even when dropped by firmware mechanisms.

- Create a WDAC base policy using `New-CIPolicy` (Windows 10 1903+)
- Set enforcement mode to **Enforced**, not Audit, in production environments
- Block unsigned PE binaries in the WPBT execution path
- Deploy via GPO: Computer Configuration › Windows Settings › Security Settings › WDAC
- Monitor `Microsoft-Windows-CodeIntegrity/Operational` for policy violations

6.3 Firmware Update Hygiene

- Source firmware updates exclusively from official vendor sites or LVFS
- Validate the SHA-256 hash of the firmware binary before flashing
- Prefer offline (pre-boot) update mode over OS-based flashing utilities
- Enable a strong UEFI setup password; disable physical DMA ports where not required
- Apply Secure Boot DBX revocation updates promptly — these revoke exploitable signed bootloaders
- Track firmware versions across the fleet using SCCM, Intune, or osquery for drift detection

07 Case Studies

7.1 LoJax — APT28 (Sednit / Fancy Bear)

Attribute	Detail
Discovery	2018 — ESET Research; first UEFI rootkit confirmed in the wild
Threat actor	APT28 (Sednit, Fancy Bear) — attributed to Russian GRU military intelligence
Targets	Government organizations in the Balkans and Central-Eastern Europe
Technique	Modified SPI flash firmware image built on a repurposed Absolute Computrace/LoJack module; survives disk replacement
Payload	Ring-0 agent drops Win32/XAgent user-space malware for C2 communication
Detection	SPI flash integrity check; unexpected module present in the firmware image

> **Significance:** LoJax is the clearest demonstration that an OEM persistence module is dual-use infrastructure. The technique did not need to be invented — it was already shipping from the factory.

7.2 MosaicRegressor — Unattributed APT-linked actor

Discovered in 2020 by Kaspersky GReAT; attribution remains unconfirmed. MosaicRegressor used a modified version of the leaked Hacking Team Vector-EDK UEFI implant framework. Targets included NGOs and diplomatic entities in Asia and Africa. The implant dropped a multi-stage downloader into the Windows filesystem at each boot.

7.3 MoonBounce — APT41

Attribute	Detail
Discovery	2021/2022 — Kaspersky GReAT
Threat actor	APT41 (Winnti Group) — attributed to Chinese state-sponsored actors
Technique	Modified the pre-existing `CORE_DXE` component rather than adding a new module — significantly harder to detect than LoJax
Payload	Installer dropped into user space; establishes a persistent C2 channel
Forensic evasion	No new files on the EFI System Partition; modification of a pre-existing firmware component only
Detection difficulty	Undetectable by conventional AV/EDR; requires byte-level comparison against a known-good vendor image

7.4 CosmicStrand

Documented by Kaspersky in 2022. Targets GIGABYTE and ASUS H81 consumer boards, injecting a kernel hook via the `CSMCORE` driver. Notable for demonstrating that firmware implants are not confined to enterprise hardware.

08 Timeline of Notable Firmware Attacks

Year	Campaign	Threat group	Type	Key novelty
2018	LoJax	APT28 (GRU)	UEFI rootkit	First UEFI rootkit confirmed in the wild; SPI modification survives disk replacement
2020	MosaicRegressor	Unknown (APT-linked)	UEFI rootkit	Based on the leaked Hacking Team implant; multi-stage downloader dropped at each boot
2021	MoonBounce	APT41	Component modification	Modified existing CORE_DXE; no new ESP files; highest stealth of documented UEFI implants
2022	CosmicStrand	Unknown (China-linked)	UEFI rootkit	Targets consumer GIGABYTE/ASUS boards; kernel hook via CSMCORE
2023	BlackLotus	Multiple actors	UEFI bootkit	First Secure Boot bypass on patched Windows 11 (CVE-2022-21894); sold commercially
2023	Gigabyte App Center	OEM (not malicious)	Supply chain	Firmware-embedded dropper across 271 board models fetching payloads over plain HTTP

Year	Campaign	Threat group	Type	Key novelty
2024	PKfail	—	Supply chain	Leaked AMI test Platform Keys across 800+ device models compromise the root of the Secure Boot chain
2025	HybridPetya	Unknown	Bootkit + ransomware	Exploits CVE-2024-7344 to install a bootkit before encrypting the MFT — firmware persistence combined with destruction

Trend: firmware persistence tooling continues to commercialize, lowering the barrier to entry well beyond nation-state actors. The 2023–2025 entries are the inflection — two of the four are not APT campaigns at all.

09 The 2026 Secure Boot Trust Anchor Transition

Microsoft's Secure Boot certificates issued in 2011 reach expiry in June 2026. This is the first refresh of the Secure Boot chain of trust since its introduction, and it changes the operational picture for every fleet in scope of this paper.

Certificate	Role	Consequence of expiry without replacement
Microsoft Corporation KEK CA 2011	Key Exchange Key — authorises DB and DBX updates	**Device can no longer receive new revocations. The DBX freezes permanently at its current state.**
Microsoft Windows Production PCA 2011	Signs the Windows boot chain	Newly signed boot components will not validate on un-updated firmware
Microsoft Corporation UEFI CA 2011	Third-party signing (shim, option ROMs, vendor tooling)	Third-party boot components affected on the same timeline

The core risk is not boot failure — it is revocation freeze. A device whose KEK has expired still boots normally. What it silently loses is the ability to accept future DBX entries. Every bootkit revocation published after that point never reaches the device. The failure mode is invisible from the endpoint.

9.1 Assessment Commands

```
# Windows — inspect current trust anchors
[System.Text.Encoding]::ASCII.GetString((Get-SecureBootUEFI KEK).Bytes) -match "2011|2023"
Confirm-SecureBootUEFI

# Linux
efi-readvar -v KEK
efi-readvar -v dbx | wc -l
mokutil --list-enrolled
```

9.2 Recommended Actions

- Inventory KEK certificate issue year across the fleet — 2011 versus 2023 — before the expiry window closes

- Compare each device's DBX against the current published UEFI revocation list; missing entries indicate a device that stopped receiving updates
- Verify firmware update availability per model; OEM support status determines whether a device can receive the 2023 anchors at all
- Treat end-of-support hardware that cannot accept new anchors as a separate, tracked risk population
- Note that TPM presence and BitLocker enrollment do **not** imply correct Secure Boot configuration — verify independently

> **Interaction with PKfail (CVE-2024-8105):** devices shipped with leaked AMI test Platform Keys are compromised at the root of the chain. For those models the anchor transition is an opportunity to replace the PK with a genuine production key, not merely to refresh the KEK.

10 Executive Recommendations

Priority	Action	Rationale
P1 — Immediate	Enable Secure Boot + TPM 2.0 + BitLocker	Foundational platform integrity chain; mitigates the majority of known bootkit techniques
P1 — Immediate	Block unsigned/revoked binaries via WDAC policy	Closes the WPBT abuse vector; prevents unsigned kernel driver loading
P1 — Immediate	Apply Secure Boot DBX updates and verify enforcement is actually enabled	Revokes exploitable signed bootloaders; the CVE-2023-24932 fix required opt-in activation
P1 — Immediate	Inventory KEK certificate vintage ahead of the June 2026 expiry	An expired KEK silently ends revocation delivery to the device
P2 — Short-term	Run an offline SPI analysis on a representative device sample	Detects descriptor misconfiguration, dormant OEM modules and firmware anomalies
P2 — Short-term	Deploy Sysmon with the ESP monitoring rule (§5.3)	Early warning of unsigned EFI binary writes
P2 — Short-term	Enable HVCI (Memory Integrity) on all endpoints	Blocks the kernel-stage component of BlackLotus
P3 — Medium-term	Integrate firmware persistence scenarios into IR playbooks	Responders must know to image SPI flash and inspect firmware before reimaging
P3 — Medium-term	Implement firmware version tracking via osquery, SCCM or Intune	A firmware baseline is a prerequisite for anomaly identification
P3 — Medium-term	Consider Platform Firmware Resilience for critical infrastructure devices	NIST SP 800-193 detect/protect/recover capability

11 Methodology & Limitations

11.1 Methodology

This whitepaper draws on vendor advisories, peer-reviewed and industry security research, conference proceedings, and first-party firmware analysis conducted by GGSEC. CVE identifiers, malware attributions and persistence mechanisms have been cross-checked against primary vendor and researcher sources rather than secondary reporting. Attribution claims reflect security research community consensus at the time of writing.

11.2 Limitations

- Offensive implementation detail — exploit code, SPI write procedures, bootkit loader internals — is deliberately omitted.
- Firmware-level attribution is technically difficult. Campaigns listed as 'unknown' or 'APT-linked' reflect genuine uncertainty rather than editorial omission.
- Heuristic detection findings, including those produced by GGSEC tooling, indicate items warranting manual review. They are not malware verdicts.
- Offline firmware analysis cannot establish runtime state. Secure Boot enablement, ME/CSE version and active service status require live host queries.
- CVE status information is time-sensitive. Verify against vendor advisories before operational use.

12 Conclusion & References

Firmware persistence remains one of the most advanced and difficult-to-detect attack vectors in modern cybersecurity. Threat actors ranging from nation-state APT groups to commercially motivated cybercriminals have demonstrated repeatable, operational firmware persistence capabilities. The barrier to entry continues to decrease as tooling becomes commercially available.

Unlike OS-level malware, firmware compromise operates below the visibility horizon of conventional endpoint security. It survives disk replacement, OS reinstallation, and standard forensic imaging. Remediation frequently requires physical component replacement or factory reflash — a reality that incident response playbooks must explicitly address.

Secure Boot, TPM, HVCI and WDAC — deployed together and maintained with current DBX revocation data — represent the minimum viable firmware security baseline for any enterprise environment in 2026. With the trust anchor transition underway, maintaining that baseline now requires active verification rather than one-time configuration. Firmware integrity must be treated as a strategic security requirement, not a niche hardening measure.

References

1. ESET Research / Smolár, M. (2023). "BlackLotus UEFI bootkit: Myth confirmed." WeLiveSecurity, 1 March 2023.
2. Microsoft Security Response Center. (2023). "Guidance for investigating attacks using CVE-2022-21894: The BlackLotus campaign."
3. NSA. (2023). "BlackLotus Mitigation Guide." Cybersecurity Information Sheet, June 2023.
4. Kaspersky GReAT. (2022). "MoonBounce: The Dark Side of UEFI Firmware."

5. Kaspersky GReAT. (2020). “MosaicRegressor: Lurking in the Shadows of UEFI.”
6. Kamluk, V., Belov, S., Sacco, A. (2014). “Absolute Backdoor Revisited.” Black Hat USA.
7. ESET Research. (2018). “LoJax: First UEFI rootkit found in the wild.”
8. Eclypsium. (2023). “Supply Chain Risk from Gigabyte App Center Backdoor.”
9. Binarly REsearch. (2024). “PKfail: Untrusted Platform Keys undermine Secure Boot” (CVE-2024-8105).
10. MITRE ATT&CK. Pre-OS Boot (T1542); Bootkit (T1542.003); System Firmware (T1542.001).
11. NIST. (2018). SP 800-193: Platform Firmware Resiliency Guidelines.
12. UEFI Forum. UEFI Specification 2.10 (2022).
13. Microsoft. Windows Defender Application Control (WDAC) documentation.
14. Intel. CHIPSEC: Platform Security Assessment Framework.

Maciej Gojny · GG Advanced IT Security · ggsec.de
GGSEC Research · Version 1.2 · July 2026 · TLP:WHITE — Public Release